

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:3

Total Marks 30

Duration :60 Minutes

Annexure A

Experiments

Code of Conduct:

I PAVAN JANA (192472313) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Pavanj

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

1. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

2. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

3. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

Experiment 1: Harris Corner Detection Analysis

Aim

To detect corner points in an image using the Harris Corner Detection algorithm and analyze the detected corners.

Procedure

1. Import the required libraries (cv2 and numpy).
2. Read the input image (image.jpg).
3. Convert the image to grayscale.
4. Convert the grayscale image to float32.
5. Apply the Harris Corner Detection algorithm.
6. Dilate the detected corners for better visibility.
7. Mark the detected corners in red.
8. Display the output image.

Code

```
harris.py - C:/Users/sreel/OneDrive/Desktop/harris.py (3.14.3)
File Edit Format Run Options Window Help
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = np.float32(gray)

corners = cv2.cornerHarris(gray, 2, 3, 0.04)

corners = cv2.dilate(corners, None)

img[corners > 0.01 * corners.max()] = [0, 0, 255]

cv2.imshow("Harris Corners", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

The Harris Corner Detection algorithm successfully detected significant corner points in the image. The detected corners represent regions with high intensity variation, making them useful for feature detection, object recognition, and image matching.

Experiment 2: Effect of Noise on Harris Corner Detection

Aim

To study the effect of Gaussian noise on Harris Corner Detection and compare the corner detection results before and after adding noise.

Procedure

1. Import the required libraries.
2. Read the input image (image.jpg).
3. Convert the image to grayscale.
4. Apply Harris Corner Detection on the original image.
5. Add Gaussian noise to the grayscale image.

6. Apply Harris Corner Detection on the noisy image.
7. Display the original image with corners, noisy image, and noisy image with detected corners.
8. Compare the results.

Code

```
harris.py - C:/Users/sreel/OneDrive/Desktop/harris.py (3.14.3)
File Edit Format Run Options Window Help
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = np.float32(gray)

corners = cv2.cornerHarris(gray, 2, 3, 0.04)

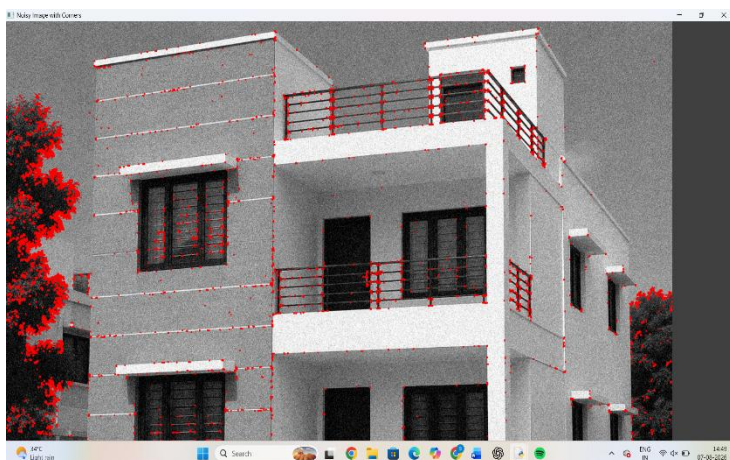
corners = cv2.dilate(corners, None)

img[corners > 0.01 * corners.max()] = [0, 0, 255]

cv2.imshow("Harris Corners", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

The Harris Corner Detection algorithm detected more false corner points after noise was added. Noise reduces the accuracy of corner detection by introducing unwanted intensity variations. Image smoothing techniques can improve detection performance.

Experiment 3: Hough Transform for Shape Detection

Aim

To detect straight lines in an image using the Hough Transform and analyze the detected line segments.

Procedure

1. Import the required libraries.
2. Read the input image (image.jpg).
3. Convert the image to grayscale.
4. Apply the Canny Edge Detection algorithm.
5. Apply the Hough Line Transform to detect straight lines.
6. Draw the detected lines on the original image.
7. Display the original image, edge image, and detected line image.
8. Analyze the detected lines.

Code

```
harris.py - C:/Users/sreel/OneDrive/Desktop/harris.py (3.14.3)
File Edit Format Run Options Window Help
import cv2
import numpy as np

# Read image
img = cv2.imread("image.jpg")

if img is None:
    print("Image not found!")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ----- Harris Corner Detection on Original Image -----
gray_float = np.float32(gray)

corners_original = cv2.cornerHarris(gray_float, 2, 3, 0.04)
corners_original = cv2.dilate(corners_original, None)

original = img.copy()
original[corners_original > 0.01 * corners_original.max()] = [0, 0, 255]

# ----- Add Gaussian Noise -----
noise = np.random.normal(0, 20, gray.shape).astype(np.int16)

noisy = gray.astype(np.int16) + noise
noisy = np.clip(noisy, 0, 255).astype(np.uint8)

# ----- Harris Corner Detection on Noisy Image -----
gray_noisy = np.float32(noisy)

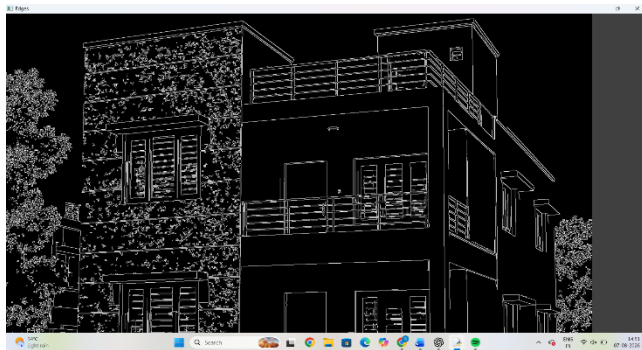
corners_noisy = cv2.cornerHarris(gray_noisy, 2, 3, 0.04)
corners_noisy = cv2.dilate(corners_noisy, None)

noisy_color = cv2.cvtColor(noisy, cv2.COLOR_GRAY2BGR)
noisy_color[corners_noisy > 0.01 * corners_noisy.max()] = [0, 0, 255]

# ----- Display Results -----
cv2.imshow("Original Image with Corners", original)
cv2.imshow("Noisy Image", noisy)
cv2.imshow("Noisy Image with Corners", noisy_color)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

The Hough Transform successfully detected the prominent straight lines in the house image, including the roof, windows, railings, and wall boundaries. The algorithm effectively identifies linear features and is widely used for shape detection in computer vision.